The system implements a **5-step pipeline** designed to automate the verification workflow for hardware engineers:

1. **Requirement Intake**: Accepts natural language descriptions and corresponding Verilog code.
2. **Test Pattern Generation**: Uses an LLM to generate intelligent test patterns covering corner cases, boundary values, and random sequences.
3. **Golden Model Creation**: Automatically generates a Python-based reference (golden) model from the natural language description to calculate expected outputs.
4. **Verification Logic Integration**: Updates the Verilog testbench with automated pass/fail checking logic based on the golden model's results.
5. **Simulation & Reporting**: Executes the simulation (e.g., via Icarus Verilog) and generates a detailed test summary.

Key Components and Technical Implementation

LLM Client (LLMClient)

The core of the system is a unified interface for interacting with LLM APIs. It supports providers like OpenAI and is configured to handle complex prompts with specific system instructions to ensure high-quality code generation.

Testbench Generator (TestbenchGenerator)

This module extracts module metadata (ports, names) from Verilog code and prompts the LLM to create a "skeleton" testbench. The LLM is instructed to focus on comprehensive coverage, including:

- Corner cases and boundary values.
- Systematic numbering of test cases.
- Input signal displays for debugging.

Golden Model Generator (GoldenModelGenerator)

A unique feature of this system is the generation of a **Python Golden Model**. By converting hardware logic into executable Python functions, the system can programmatically determine the "correct" answer for any given input pattern without requiring manual entry from the user.

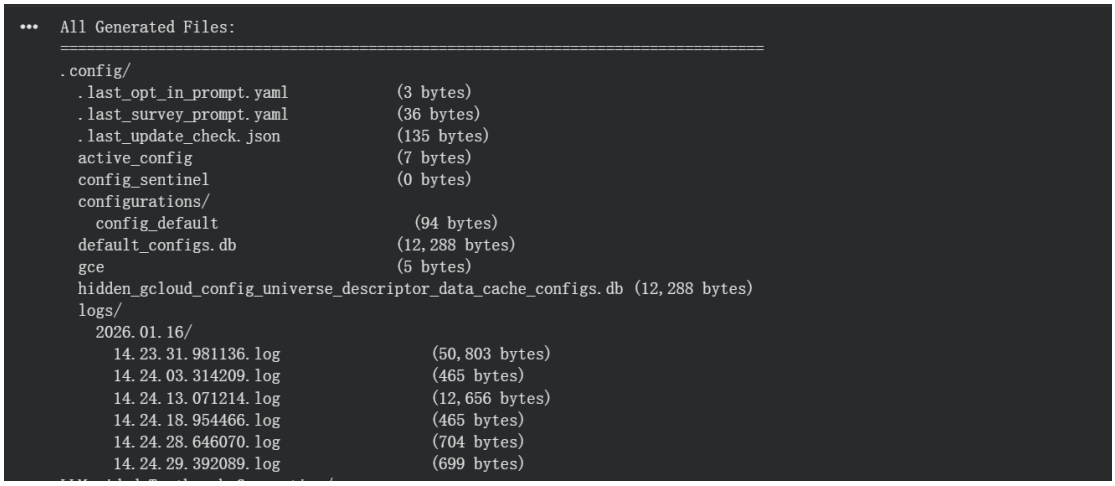
Testbench Updater (TestbenchUpdater)

The final stage involves injecting verification logic into the generated testbench. It adds:

- **Result Tracking**: Variables like passed_tests and failed_tests.
- **Comparison Logic**: Automated checks that compare actual simulation outputs against the golden outputs.
- **Visual Feedback**: Uses symbols like "✓" and "✗" for clear reporting.

Features & Benefits

- **Automation**: Reduces the manual effort required to write complex verification environments.
- **Accuracy**: Uses a secondary Python implementation (Golden Model) to verify the primary Verilog logic.
- **Self-Contained**: The provided environment is ready-to-run, requiring only standard tools like iverilog and an OpenAI API key.
- **Intelligent Coverage**: Leverages GPT-4's reasoning to identify edge cases that might be overlooked by human designers.



```
LLM-aided-Testbench-Generation/
  .git/
    HEAD                                (21 bytes)
    branches/
    config                              (284 bytes)
    description                         (73 bytes)
    hooks/
      applypatch-msg.sample               (478 bytes)
      commit-msg.sample                   (896 bytes)
      fsmonitor-watchman.sample           (4,655 bytes)
      post-update.sample                  (189 bytes)
      pre-applypatch.sample               (424 bytes)
      pre-commit.sample                   (1,643 bytes)
      pre-merge-commit.sample             (416 bytes)
      pre-push.sample                     (1,374 bytes)
      pre-rebase.sample                   (4,898 bytes)
      pre-receive.sample                  (544 bytes)
      prepare-commit-msg.sample           (1,492 bytes)
      push-to-checkout.sample             (2,783 bytes)
      update.sample                       (3,650 bytes)
    index                               (2,387 bytes)
    info/
      exclude                             (240 bytes)
    logs/
      HEAD                                (207 bytes)
      refs/
        heads/
          main                                (207 bytes)
        remotes/
          origin/
            HEAD                                (207 bytes)
    objects/
      info/
      pack/
```

Then generate a custom module（in my case is 4to2 priority encoder）

```
[Step 5] Updating testbench with golden outputs and verification logic...
  - Saved final testbench to: priority_encoder/testbench_final.v

================================================================================
Pipeline completed successfully!
================================================================================

Generated files in 'priority_encoder':
  - testbench_initial.v    : Initial testbench with test patterns
  - golden_model.py        : Python reference implementation
  - test_patterns_with_golden.json : Test patterns with expected outputs
  - testbench_final.v      : Final testbench with verification


================================================================================
CUSTOM MODULE PROCESSING COMPLETE
================================================================================
```

Use the pipeline to get a testbench

Then put a buggy design in it

```
#4. Buggy design
buggy_pe_verilog = """
module priority_encoder_4to2 (
        input [3:0] in,
        output reg [1:0] out,
        output reg valid
);
        always @(*) begin
                valid = 1'b1;
                if (in[0])              out = 2'b00; // BUG
                else if (in[1]) out = 2'b01;
                else if (in[2]) out = 2'b10;
                else if (in[3]) out = 2'b11; // BUG:
                else begin
                        out = 2'b00;
                        valid = 1'b0;
                end
        end
endmodule
"""
```

Turned out it can find the bug

```
  Simulation Output:
========================================
Test 1: in=0000
✓ Test 1 Passed: out=00, valid=0
Test 2: in=0001
✓ Test 2 Passed: out=00, valid=1
Test 3: in=0010
✓ Test 3 Passed: out=01, valid=1
Test 4: in=0100
✓ Test 4 Passed: out=10, valid=1
Test 5: in=1000
✓ Test 5 Passed: out=11, valid=1
Test 6: in=0011
✗ Test 6 Failed: Expected out=01, valid=1, Got out=00, valid=1
Test 7: in=0110
✗ Test 7 Failed: Expected out=10, valid=1, Got out=01, valid=1
Test 8: in=1100
✗ Test 8 Failed: Expected out=11, valid=1, Got out=10, valid=1
Test 9: in=1111
✗ Test 9 Failed: Expected out=11, valid=1, Got out=00, valid=1
Test 10: in=1010
✗ Test 10 Failed: Expected out=11, valid=1, Got out=01, valid=1
Test 11: in=0101
✗ Test 11 Failed: Expected out=10, valid=1, Got out=00, valid=1
Test Summary:
Total Tests Run: 22
Tests Passed: 10
Tests Failed: 12

========================================

[RESULT] Verification Successful: The testbench correctly detected the priority logic bug! ✗
```

```
----------------------------------------
>>> Step 2: Simulating FIXED code (Confirming pass after fix)...
  Simulating priority_encoder_4to2.v with priority_encoder/testbench_final.v...
  Compilation successful
  Simulation Output:
========================================
Test 1: in=0000
✓ Test 1 Passed: out=00, valid=0
Test 2: in=0001
✓ Test 2 Passed: out=00, valid=1
Test 3: in=0010
✓ Test 3 Passed: out=01, valid=1
Test 4: in=0100
✓ Test 4 Passed: out=10, valid=1
Test 5: in=1000
✓ Test 5 Passed: out=11, valid=1
Test 6: in=0011
✓ Test 6 Passed: out=01, valid=1
Test 7: in=0110
✓ Test 7 Passed: out=10, valid=1
Test 8: in=1100
✓ Test 8 Passed: out=11, valid=1
Test 9: in=1111
✓ Test 9 Passed: out=11, valid=1
Test 10: in=1010
✓ Test 10 Passed: out=11, valid=1
Test 11: in=0101
✓ Test 11 Passed: out=10, valid=1
Test Summary:
Total Tests Run: 22
Tests Passed: 22
Tests Failed: 0

========================================

[RESULT] Verification Successful: The fixed design passed all tests! ✓
```

```
LLM-aided-Testbench-Generation/
  .git/
    HEAD                                (21 bytes)
    branches/
    config                              (284 bytes)
    description                         (73 bytes)
    hooks/
      applypatch-msg.sample               (478 bytes)
      commit-msg.sample                   (896 bytes)
      fsmonitor-watchman.sample           (4,655 bytes)
      post-update.sample                  (189 bytes)
      pre-applypatch.sample               (424 bytes)
      pre-commit.sample                   (1,643 bytes)
      pre-merge-commit.sample             (416 bytes)
      pre-push.sample                     (1,374 bytes)
      pre-rebase.sample                   (4,898 bytes)
      pre-receive.sample                  (544 bytes)
      prepare-commit-msg.sample           (1,492 bytes)
      push-to-checkout.sample             (2,783 bytes)
      update.sample                       (3,650 bytes)
    index                               (2,387 bytes)
    info/
      exclude                             (240 bytes)
    logs/
      HEAD                                (207 bytes)
      refs/
        heads/
          main                                (207 bytes)
        remotes/
          origin/
            HEAD                                (207 bytes)
    objects/
      info/
      pack/
```